Day17 完整GPT模型结构定义（手写小GPT计划）

一、Day17 核心目标（贴合周3计划）

本周核心任务是“堆叠Transformer Block，完成GPT结构”，Day17作为关键节点，需实现：

1. 整合前期所有组件（Embedding、位置编码、Transformer Block等），封装成标准 `nn.Module` 类
2. 严格遵循GPT结构规范：Pre-LN架构 + 因果掩码 + 残差连接 + 输出头
3. 确保模型可前向推理，实现“token_ids输入 → logits输出”的完整流程
4. 为Day18模型前向测试、Day21 `model.generate()` 雏形铺垫

二、前期组件回顾（周1+周2已完成，直接复用）

Day17无需重复手写基础组件，复用周2已实现的核心模块，确保组件兼容性：

- Scaled Dot-Product Attention（Day8）
- Multi-Head Attention（Day10）
- LayerNorm、Dropout、残差连接（Day11）
- Feed Forward（FFN）网络（Day12）
- Transformer Block（Day15，Pre-LN架构）
- Embedding层 + 位置编码（Day16，含sin/cos固定PE、可学习PE）
- Mask机制（Day9，因果掩码+padding掩码）

三、Day17 核心：完整GPT模型定义（可直接运行）

基于PyTorch手写，无高级库依赖，结构对齐GPT-2，适配前期所有组件，可直接用于后续推理和生成测试。

```python
import torch
import torch.nn as nn
import torch.nn.functional as F
import math
import os

# 适配Mac MPS/GPU/CPU，贴合周1Day3 CUDA基础
```

```python
os.environ["PYTORCH_ENABLE_MPS_FALLBACK"] = "1"

# ---------------------------------------------------------------------------
# 复用周2组件：1. Mask机制（因果掩码 + padding掩码）（Day9）
# ---------------------------------------------------------------------------
def create_padding_mask(seq, pad_idx=0):
    """padding掩码：屏蔽pad_idx对应的位置，shape [B, 1, 1, T]"""
    mask = (seq == pad_idx)
    return mask.unsqueeze(1).unsqueeze(2)

def create_causal_mask(seq_len):
    """因果掩码：屏蔽未来位置，确保生成时看不到后续token，shape [1, 1, T, T]"""
    mask = torch.tril(torch.ones(seq_len, seq_len, dtype=torch.bool))
    mask = ~mask  # True=需要屏蔽，False=可访问
    return mask.unsqueeze(0).unsqueeze(1)

# ---------------------------------------------------------------------------
# 复用周2组件：2. Scaled Dot-Product Attention（Day8）
# ---------------------------------------------------------------------------
class ScaledDotProductAttention(nn.Module):
    def __init__(self, dropout=0.1):
        super().__init__()
        self.dropout = nn.Dropout(dropout)

    def forward(self, q, k, v, mask=None):
        d_k = q.size(-1)
        # 注意力分数计算：Q·K^T / √d_k（防止梯度消失/爆炸）
        scores = torch.matmul(q, k.transpose(-2, -1)) / math.sqrt(d_k)
        # 应用掩码（因果+padding）
        if mask is not None:
            scores = scores.masked_fill(mask, -1e9)
        # 注意力权重归一化 + dropout
        attn_weights = F.softmax(scores, dim=-1)
        attn_weights = self.dropout(attn_weights)
        # 注意力输出：权重 × 值
        output = torch.matmul(attn_weights, v)
        return output, attn_weights

# ---------------------------------------------------------------------------
# 复用周2组件：3. Multi-Head Attention（Day10）
# ---------------------------------------------------------------------------
```

```python
class MultiHeadAttention(nn.Module):
    def __init__(self, d_model, n_heads, dropout=0.1):
        super().__init__()
        assert d_model % n_heads == 0, "d_model必须能被n_heads整除"
        self.d_model = d_model
        self.n_heads = n_heads
        self.head_dim = d_model // n_heads  # 每个头的维度

        # 线性投影层（Q、K、V分别投影）
        self.wq = nn.Linear(d_model, d_model)
        self.wk = nn.Linear(d_model, d_model)
        self.wv = nn.Linear(d_model, d_model)
        # 输出投影层
        self.wo = nn.Linear(d_model, d_model)
        # 注意力模块
        self.attn = ScaledDotProductAttention(dropout)
        self.dropout = nn.Dropout(dropout)

    def split_heads(self, x):
        """将输入拆分到多个头：[B, T, d_model] → [B, n_heads, T, head_dim]"""
        B, T, C = x.shape
        return x.view(B, T, self.n_heads, self.head_dim).transpose(1, 2)

    def forward(self, q, k, v, mask=None):
        B, T, C = q.shape
        # 1. 线性投影
        q = self.wq(q)
        k = self.wk(k)
        v = self.wv(v)
        # 2. 拆分多头
        q = self.split_heads(q)
        k = self.split_heads(k)
        v = self.split_heads(v)
        # 3. 计算注意力
        attn_out, _ = self.attn(q, k, v, mask)
        # 4. 拼接多头输出，恢复原形状
        attn_out = attn_out.transpose(1, 2).contiguous().view(B, T, C)
        # 5. 输出投影 + dropout
        return self.dropout(self.wo(attn_out))

# ---------------------------------------------------------------------------
# 复用周2组件：4. Feed Forward（FFN）网络（Day12，修正dropout位置）
# ---------------------------------------------------------------------------
class FeedForward(nn.Module):
    def __init__(self, d_model, dropout=0.1, expansion=4):
```

```python
        super().__init__()
        # FFN核心结构：升维→激活→dropout→降维
        self.fc1 = nn.Linear(d_model, d_model * expansion)  # 升维（d_model→4*d_model）
        self.fc2 = nn.Linear(d_model * expansion, d_model)  # 降维（4*d_model→d_model）
        self.gelu = nn.GELU()  # GPT标配激活函数
        self.dropout = nn.Dropout(dropout)  # 中间dropout（激活后、降维前）

    def forward(self, x):
        return self.fc2(self.dropout(self.gelu(self.fc1(x))))

# ---------------------------------------------------------------------------
# 复用周2组件：5. Transformer Block（Pre-LN架构，Day15）
# ---------------------------------------------------------------------------
class TransformerBlock(nn.Module):
    def __init__(self, d_model, n_heads, dropout=0.1):
        super().__init__()
        self.attn = MultiHeadAttention(d_model, n_heads, dropout)
        self.ffn = FeedForward(d_model, dropout)
        # Pre-LN：先归一化，再进行子层运算（深层训练更稳定）
        self.norm1 = nn.LayerNorm(d_model)
        self.norm2 = nn.LayerNorm(d_model)
        self.dropout = nn.Dropout(dropout)

    def forward(self, x, mask=None):
        # 注意力子层：Norm → Attention → 残差连接
        residual = x
        x = self.norm1(x)
        x = residual + self.dropout(self.attn(x, x, x, mask))  # 自注意力（Q=K=V）

        # FFN子层：Norm → FFN → 残差连接
        residual = x
        x = self.norm2(x)
        x = residual + self.dropout(self.ffn(x))
        return x

# ---------------------------------------------------------------------------
# 复用周3组件：6. Embedding + 位置编码（Day16）
# ---------------------------------------------------------------------------
# 6.1 词嵌入层（nn.Embedding，Day16重点讲解）
class TokenEmbedding(nn.Module):
```

```python
    def __init__(self, vocab_size, d_model, pad_idx=0):
        super().__init__()
        # 核心：[vocab_size, d_model]可训练矩阵，实现“token_id→向量”查表
        self.embedding = nn.Embedding(vocab_size, d_model, padding_idx=pad_idx)
        self.scale = math.sqrt(d_model)  # 缩放因子，稳定嵌入层输出

    def forward(self, x):
        # 输入：[B, T]（token_ids），输出：[B, T, d_model]（词嵌入向量）
        return self.embedding(x) * self.scale

# 6.2 可学习位置编码（GPT标配，Day16重点讲解）
class LearnablePE(nn.Module):
    def __init__(self, d_model, max_len=512, dropout=0.1):
        super().__init__()
        # 可训练位置编码矩阵：[1, max_len, d_model]，每个位置对应d_model维向量
        self.pe = nn.Parameter(torch.randn(1, max_len, d_model))
        self.dropout = nn.Dropout(dropout)

    def forward(self, x):
        B, T, C = x.shape
        # 截取当前序列长度的位置编码，与词嵌入相加（广播适配batch）
        x = x + self.pe[:, :T]
        return self.dropout(x)

# 6.3 sin/cos固定位置编码（备用，Day16讲解，可替换使用）
class SinCosPE(nn.Module):
    def __init__(self, d_model, max_len=512, dropout=0.1):
        super().__init__()
        self.dropout = nn.Dropout(dropout)
        # 初始化位置编码矩阵：[max_len, d_model]
        pe = torch.zeros(max_len, d_model)
        # 生成位置索引：[max_len, 1]
        position = torch.arange(max_len).unsqueeze(1)
        # 计算衰减因子：[d_model//2]，多尺度频率
        div_term = torch.exp(torch.arange(0, d_model, 2) * (-math.log(10000.0) / d_model))
        # 奇偶位分别用sin、cos编码
        pe[:, 0::2] = torch.sin(position * div_term)
        pe[:, 1::2] = torch.cos(position * div_term)
        # 注册为缓冲区（不参与训练，随模型保存）
        self.register_buffer('pe', pe)

    def forward(self, x):
        B, T, C = x.shape
        x = x + self.pe[:T]
        return self.dropout(x)
```

```python

# ---------------------------------------------------------------------------
# 🔥 Day17 核心：完整GPT模型（整合所有组件）
# ---------------------------------------------------------------------------
class MiniGPT(nn.Module):
    def __init__(
        self,
        vocab_size,          # 词表大小（周1Day4-5已实现词表构建）
        d_model=512,         # 模型特征维度（所有组件统一维度）
        n_heads=8,           # 多头注意力的头数
        n_layers=6,          # Transformer Block堆叠层数
        max_len=512,         # 最大序列长度（适配位置编码）
        dropout=0.1,         # dropout概率（正则化）
        pad_idx=0,           # padding符号的token_id
        use_learnable_pe=True  # 选择位置编码类型（可学习/固定）
    ):
        super().__init__()
        self.pad_idx = pad_idx
        self.max_len = max_len
        self.d_model = d_model

        # 1. 词嵌入层（输入层：token_id→向量）
        self.token_emb = TokenEmbedding(vocab_size, d_model, pad_idx)
        # 2. 位置编码层（注入时序信息）
        if use_learnable_pe:
            self.pos_emb = LearnablePE(d_model, max_len, dropout)
        else:
            self.pos_emb = SinCosPE(d_model, max_len, dropout)

        # 3. Transformer Block堆叠（核心网络）
        self.blocks = nn.ModuleList([
            TransformerBlock(d_model, n_heads, dropout)
            for _ in range(n_layers)
        ])

        # 4. 输出层（GPT生成任务：向量→词表概率）
        self.final_norm = nn.LayerNorm(d_model)  # 最终归一化
        self.lm_head = nn.Linear(d_model, vocab_size, bias=False)  # 映射回词表

        # 权重绑定（GPT标配）：词嵌入权重与输出层权重共享，减少参数量
        self.token_emb.embedding.weight = self.lm_head.weight

        # 初始化参数（保证训练稳定性）
        self._init_weights()

```

```python
    def _init_weights(self):
        """参数初始化：避免梯度消失/爆炸，适配GPT训练"""
        for m in self.modules():
            if isinstance(m, nn.Linear):
                nn.init.normal_(m.weight, mean=0.0, std=0.02)
                if m.bias is not None:
                    nn.init.zeros_(m.bias)
            elif isinstance(m, nn.Embedding):
                nn.init.normal_(m.weight, mean=0.0, std=0.02)

    def forward(self, idx):
        """
        完整前向推理流程（贴合Day17目标）
        输入：idx → [B, T]（batch_size, seq_len），token_ids序列
        输出：logits → [B, T, vocab_size]，每个位置的词表概率分布
        """
        B, T = idx.shape
        # 校验序列长度（不超过max_len）
        assert T <= self.max_len, f"序列长度{T}超过模型最大长度{self.max_len}"

        # 1. 生成掩码（因果掩码+padding掩码，结合使用）
        pad_mask = create_padding_mask(idx, self.pad_idx).to(idx.device)
        causal_mask = create_causal_mask(T).to(idx.device)
        mask = pad_mask | causal_mask  # 合并掩码，同时屏蔽padding和未来位置

        # 2. 词嵌入 + 位置编码（输入层：注入语义+时序信息）
        x = self.token_emb(idx)
        x = self.pos_emb(x)

        # 3. 经过所有Transformer Block（核心特征提取）
        for block in self.blocks:
            x = block(x, mask)

        # 4. 最终归一化 + 输出logits（用于后续预测/生成）
        x = self.final_norm(x)
        logits = self.lm_head(x)

        return logits

# ---------------------------------------------------------------------------
# Day17 验收测试（贴合周测要求，验证模型可运行）
# ---------------------------------------------------------------------------
if __name__ == "__main__":
    # 1. 设备配置（适配周1Day3 CUDA/MPS基础）
    device = torch.device(
```

```python
        "cuda" if torch.cuda.is_available() else
        "mps" if torch.backends.mps.is_available() else "cpu"
    )
    print(f"使用设备：{device}")

    # 2. 超参数配置（适配前期组件，简化版便于测试）
    vocab_size = 1000  # 周1Day4已实现简单词表，此处复用
    d_model = 128      # 简化维度，加快测试速度
    n_heads = 4        # 多头注意力头数（d_model需能被n_heads整除）
    n_layers = 2       # 简化堆叠层数，便于前向测试
    max_len = 64       # 最大序列长度

    # 3. 初始化模型
    model = MiniGPT(
        vocab_size=vocab_size,
        d_model=d_model,
        n_heads=n_heads,
        n_layers=n_layers,
        max_len=max_len,
        use_learnable_pe=True  # 使用可学习位置编码（GPT标配）
    ).to(device)

    # 4. 构造输入（模拟周1Day6的GPT专用数据格式：input序列）
    # 输入shape：[B, T] = [2, 10]（batch=2, seq_len=10）
    idx = torch.randint(1, vocab_size, (2, 10)).to(device)  # 避开pad_idx=0

    # 5. 前向推理测试（无梯度计算，快速验证）
    with torch.no_grad():
        logits = model(idx)

    # 6. 验收标准（Day17核心要求）
    print(f"输入token_ids shape：{idx.shape} → [batch_size, seq_len]")
    print(f"输出logits shape：{logits.shape} → [batch_size, seq_len, vocab_size]")
    print("✅ Day17 完整GPT模型前向推理通过！")
    print("✅ 已完成Day17目标：完整GPT模型结构定义，可用于后续前向测试和生成任务！")

```

四、Day17 关键要点（贴合计划，重点掌握）

1. 模型结构逻辑（GPT标准流程）

输入 → Token Embedding（语义） → 位置编码（时序） → 堆叠Transformer Block（特征提取） → 最终归一化 → 输出logits（词表概率）

2. 核心细节说明（适配前期组件，避免踩坑）

- 权重绑定：`token_emb.embedding.weight = lm_head.weight`，GPT标配，减少参数量，稳定训练（周2组件延伸）
- 掩码机制：因果掩码+padding掩码合并使用，确保生成时“看不到未来、屏蔽无效padding”（Day9重点）
- 位置编码可选：支持可学习PE（GPT标配）和sin/cos固定PE（Day16讲解），可通过`use_learnable_pe`切换
- 参数初始化：自定义初始化逻辑，避免梯度消失/爆炸，适配深层Transformer堆叠（Day11 Dropout/LayerNorm延伸）

3. 与计划的衔接说明

- 复用周1内容：设备配置（CUDA/MPS）、词表大小（vocab_size）、输入格式（token_ids）
- 复用周2内容：所有Attention、FFN、Transformer Block组件，确保代码兼容性
- 为Day18铺垫：模型可正常前向推理，输出logits，后续可直接用于“前向测试+损失计算”
- 为Day21铺垫：模型结构完整，后续只需添加`generate`方法，即可实现文本生成

五、Day17 验收标准（必过）

1. 运行代码不报错，成功打印设备信息、输入/输出shape
2. 输入shape为`[B, T]`，输出logits shape为`[B, T, vocab_size]`（贴合GPT生成任务需求）
3. 理解模型各组件的衔接逻辑，能对应到周1、周2的具体Day内容
4. 可自由切换位置编码类型（可学习/固定），验证组件兼容性

> 💡 Day17小结：本周核心是“堆叠组件、完成GPT结构”，今日已实现完整模型封装，后续Day18-21将围绕“测试、优化、生成”展开，逐步实现“手写小GPT，能生成文本”的最终目标。

（注：文档部分内容可能由 AI 生成）